

UCS503 - Software Engineering

Elevate Load Tester

An Automated Input Analysis and Asynchronous Load Testing Platform

Submitted by:

102403671 Yash Bharadwaj

1024030672 Nikhil Khosla

1024031155 Krish Agrawal

Group: 3C51 BE Third Year, CSE

Submitted to:

Dr. Raghav B. Venkataramaiyer

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	5
1.1	Background and Context	5
1.2	Problem Statement	5
1.3	Project Objectives	6
2	Methodology and Design	7
2.1	System Architecture	7
2.2	Frontend Interface	8
2.3	Target Analysis and Validation	9
2.4	Automated Edge-Case Generation	9
2.5	Asynchronous Load-Test Execution	11
2.6	Result Collection and Presentation	11
2.7	Run Comparison	12
2.8	Containerization and Deployment Architecture	13
2.9	CI/CD Workflow	14
2.10	Technical Stack	14
3	Implementation and Evaluation	15
3.1	Implementation	15
3.2	Target Analysis Workflow	15
3.3	Test-Case Construction	16
3.4	Load-Test Execution	16
3.5	Result Processing	16
3.6	Evaluation Methodology	16
3.7	Evaluation Metrics	17
3.7.1	Total Requests	17
3.7.2	Failures	17
3.7.3	Response Time	17
3.7.4	Requests per Second	17
3.7.5	Duration	18
3.8	Experimental Configurations	18
3.9	Results	18
3.10	Analysis of Results	18
3.11	Deployment Evaluation	19
4	Conclusion	21
4.1	Summary of Work	21
4.2	Key Findings	21

4.3	Limitations	22
4.4	Future Work and Recommendations	22
4.5	Final Remarks	23

Introduction

1.1 Background and Context

Performance and load testing are important parts of software engineering because an application that works correctly for a small number of users may behave very differently when subjected to concurrent requests. Traditional load-testing workflows commonly require a tester to manually identify an endpoint, construct representative requests, prepare test data, configure a load-testing tool, execute the test, and then interpret the resulting metrics.

The project developed by Team Elevate addresses this workflow by placing an input-analysis layer before the load-testing engine. Instead of immediately sending a target URL to the load-testing system, the platform first accepts a website or API target and analyses the target. For API targets, the system can use a sample request/input to identify potentially problematic input conditions. For website targets, the system can analyse the supplied target and support page-crawling-oriented testing.

The identified cases are presented to the user before execution. The user can select the cases that should participate in the load test and configure the user population, spawn rate, and duration. The selected workload is then submitted for asynchronous execution using Locust. The results are made available through the frontend, including request counts, failures, response-time statistics and comparison of previous test runs.

This approach turns load testing from a primarily manual configuration task into a guided workflow consisting of target validation, input analysis, test-case selection, asynchronous execution, monitoring, and result comparison.

1.2 Problem Statement

A conventional load-testing workflow may begin directly with a URL and a set of manually prepared requests. This creates several practical problems. First, the tester must understand the target interface before a meaningful workload can be constructed. Second, edge cases such as missing fields, null values, incorrect data types, boundary values, or suspicious input patterns may be overlooked. Third, repeated load tests can become difficult to compare when results are displayed only as independent execution reports.

The project therefore focuses on developing a platform that inserts an automated analysis and selection stage before load generation. The system should validate

whether a target is reachable, inspect the supplied input where applicable, generate useful test cases, allow the user to select the desired cases, execute the workload asynchronously, and present the resulting performance metrics in a form that supports interpretation and comparison.

The resulting system is intended as a practical demonstration of the integration of web application development, backend API design, automated test-case generation, load testing, containerized execution, Kubernetes-based orchestration, cloud deployment, and CI/CD.

1.3 Project Objectives

The main objectives of the project are:

1. Develop a web-based interface through which a user can submit a website URL or API target for analysis.
2. Perform a preliminary reachability and sanity check before initiating a load test.
3. Analyse sample API input and identify useful edge cases for testing.
4. Present generated test cases to the user and allow selective case execution.
5. Support configurable virtual-user population, spawn rate, and test duration.
6. Execute load tests asynchronously so that the frontend does not have to remain blocked while a test is running.
7. Use Locust as the load-testing engine to generate concurrent requests and collect performance measurements.
8. Run the backend and load-testing workload in a cloud-hosted environment using containerized services and lightweight Kubernetes orchestration.
9. Integrate the project workflow with CI/CD so that changes can be built, tested, and deployed through an automated pipeline.
10. Present test-session results using request counts, failures, average/minimum/maximum response times, throughput, and duration.
11. Provide comparison of previous test runs so that changes in workload or system behaviour can be evaluated more easily.

Methodology and Design

2.1 System Architecture

The platform follows a layered architecture in which the frontend communicates with the backend, while the backend coordinates target analysis, test-case generation, asynchronous execution, and result retrieval. The load-testing engine is kept separate from the user-facing interface so that long-running test sessions can execute independently.

The overall workflow is:

1. The user enters a target URL or API endpoint in the frontend.
2. The frontend sends the target information to the backend analysis service.
3. The backend performs a reachability/sanity check and determines the appropriate analysis path.
4. For API targets, sample request/input information is analysed and candidate edge cases are generated.
5. The frontend displays the generated cases and allows the user to select the cases to execute.
6. The user specifies the load configuration, including the number of users, spawn rate, and duration.
7. The backend creates an asynchronous test session and passes the selected workload to the Locust-based load-testing component.
8. The test executes in the deployed environment while the frontend can query the status of the session.
9. On completion, the backend stores or exposes the resulting metrics for display.
10. The user can inspect an individual run or compare it with a previous run.

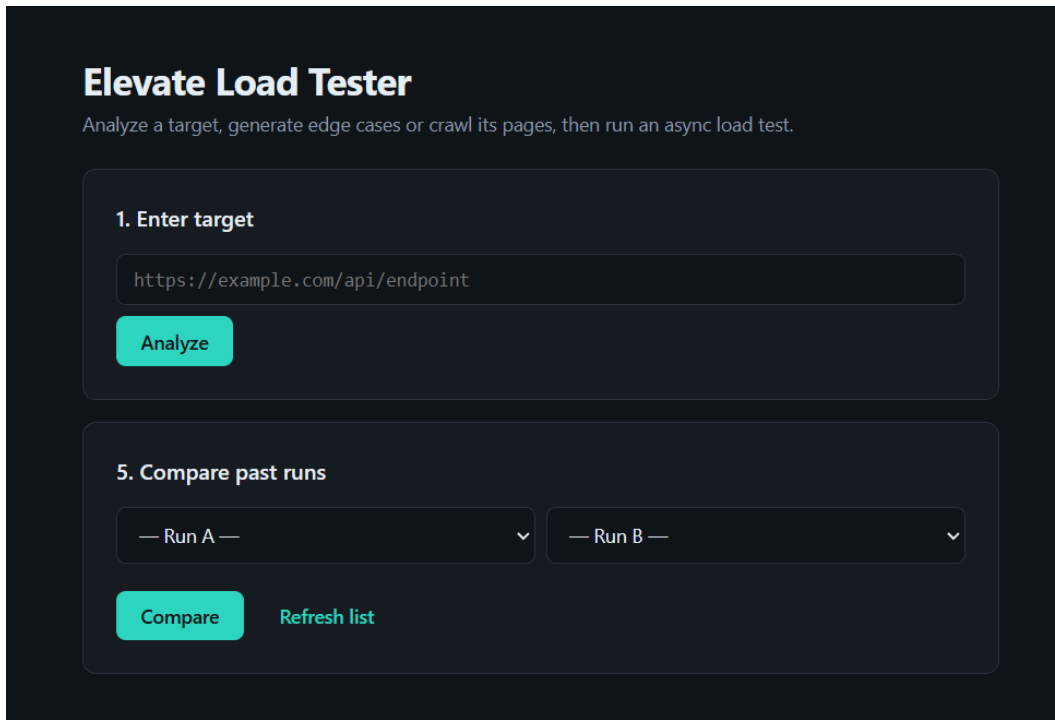


Figure 2.1: Frontend interface for target submission and test-run comparison

The frontend evidence shows that the implemented interface is organised around the analysis-to-testing workflow rather than exposing the Locust engine directly to the user. The interface contains a target-entry stage and a comparison stage, while subsequent screens support edge-case selection and test execution.

2.2 Frontend Interface

The frontend provides the primary interaction layer for the system. Its purpose is to hide implementation details of the load-testing infrastructure while still exposing the controls necessary for a meaningful test.

The main user-facing stages are:

- **Target entry:** The user enters a website or API endpoint and starts analysis.
- **Input analysis:** The system reports the information obtained from the target and, for API input, presents a sample-input workflow.
- **Edge-case selection:** Generated cases are displayed as selectable items.
- **Load configuration:** The user specifies users, spawn rate, and duration.

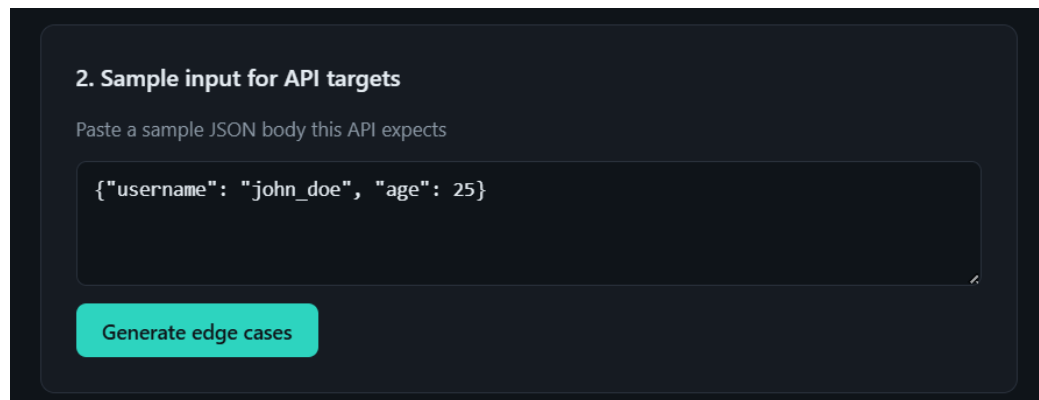
- **Load-test status:** The user can monitor a submitted test session and inspect its metrics after completion.
- **Run comparison:** Previous completed runs can be selected and compared using common performance metrics.

2.3 Target Analysis and Validation

The first backend stage is target analysis. Before a potentially expensive load test is launched, the target is checked for basic reachability and the system determines how the target should be handled.

This stage is important because load generation should not be the first mechanism used to discover that a target is unavailable or that the supplied input is unusable. Separating validation from execution also gives the user feedback before computing resources are consumed by a test session.

For API targets, the platform accepts sample request information and uses it as the basis for generating additional cases. The frontend evidence shows that the system is capable of displaying an API sample-input stage before edge-case selection.



2. Sample input for API targets

Paste a sample JSON body this API expects

```
{"username": "john_doe", "age": 25}
```

Generate edge cases

Figure 2.2: API sample-input stage of the frontend workflow

2.4 Automated Edge-Case Generation

A central feature of the project is the input-analysis layer. Rather than requiring the user to manually create every request variant, the system generates candidate cases from the supplied input.

The implemented interface demonstrates categories including:

- **Missing-field cases**, where an expected field is removed.

- **Null-value cases**, where a field is explicitly assigned a null value.
- **Type-mismatch cases**, where a value is provided using an unexpected type.
- **Boundary-value cases**, including empty strings and unusually long values.
- **Known attack-pattern cases**, such as SQL-injection-like and XSS-like input strings.

The purpose of these generated cases in the load-testing workflow is not to replace a dedicated security-testing system. Instead, they provide a broader set of input conditions that can be exercised repeatedly under load and can reveal differences in response behaviour or failure rates.

The screenshot shows a web interface titled "3. Select edge cases". It features a list of nine test cases, each with a blue checkmark icon to its left. The cases are: "[missing_field] Request with 'username' removed entirely", "[null_value] 'username' set to null", "[type_mismatch] 'username' given a int instead of str", "[boundary_value] 'username' set to empty string", "[boundary_value] 'username' set to very long string", "[known_attack] SQL injection attempt in 'username'", "[known_attack] SQL injection attempt in 'username'", "[known_attack] SQL injection attempt in 'username'", and "[known_attack] XSS attempt in 'username'". Below the list, there are three input fields: "Users (1-1000)" with the value "50", "Spawn Rate (1-20)" with the value "15", and "Duration in seconds (1-300)" with the value "10". At the bottom, there is a red button labeled "Confirm selection and start load test".

Users (1-1000)	Spawn Rate (1-20)	Duration in seconds (1-300)
50	15	10

Figure 2.3: Edge-case selection and load configuration interface

The user can select individual cases instead of being forced to execute the entire generated set. This provides control over workload composition and allows a tester to focus on the cases relevant to the target.

2.5 Asynchronous Load-Test Execution

After the user confirms the selected cases and load parameters, the backend initiates a test session. The load test is executed asynchronously so that the request which starts the test does not need to remain occupied for the entire duration of the workload.

The execution layer uses Locust as the load-testing engine. Locust is responsible for generating concurrent user activity and collecting request-level performance information. The backend acts as the orchestration layer around Locust: it prepares the selected test workload, starts the execution, tracks the session, and makes the resulting information available to the frontend.

The load configuration exposed by the frontend includes the number of users, spawn rate, and duration. These parameters allow the same target and edge-case selection to be evaluated under different levels of concurrency.

2.6 Result Collection and Presentation

The completed test session is presented using metrics that are directly useful for performance analysis. The frontend evidence includes:

- Total requests
- Total failures
- Average response time
- Minimum response time
- Maximum response time
- Requests per second
- Test duration

The interface also exposes the individual edge-case/request rows, allowing the user to see how different generated cases behaved during the same test session.

4. Load test status

Job 30da8791-d9c1-49b7-8434-7c8e98e2fc91 Completed

Edge Case	Requests	Failures	Avg (ms)	Min (ms)	Max (ms)
empty_string_username	18	0	138	114	464
missing_age	20	0	221	114	627
missing_username	16	0	188	113	782
negative_age	19	0	250	114	797
null_age	16	0	196	114	543
null_username	17	0	228	114	560
sqli_username_0	20	0	198	113	599
sqli_username_1	13	0	229	113	614
sqli_username_2	13	0	243	113	648
very_large_age	24	0	198	113	474
very_long_string_username	18	0	171	113	663
wrong_type_age	11	0	281	114	969
wrong_type_username	24	0	202	113	469
xss_username_0	21	1	174	113	571
xss_username_1	19	0	230	114	469
zero_age	25	0	203	113	471
Total	294	1	206	113	969

[Show raw response](#)

Figure 2.4: Example completed load-test result shown by the frontend

The displayed example contains 294 total requests, one reported failure, an average response time of approximately 206 ms, a minimum response time of approximately 113 ms, and a maximum response time of approximately 969 ms. These values describe the illustrated sample run only and are not treated as universal performance characteristics of the platform.

2.7 Run Comparison

A separate comparison facility allows two completed test sessions to be selected and evaluated side by side. This is useful when the same target is tested with different user populations, edge-case selections, configurations, or software versions.

The comparison view presents common metrics in a tabular form. The supplied frontend evidence demonstrates comparison of total requests, total failures, average response time, requests per second, and duration.

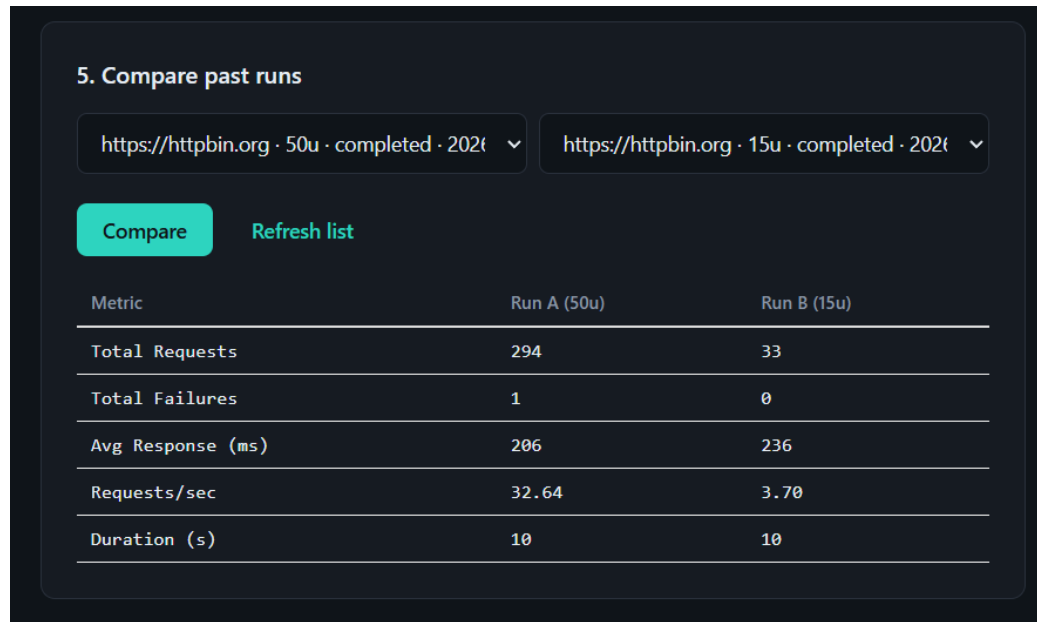


Figure 2.5: Comparison of two completed test runs

In the illustrated comparison, Run A contains 294 requests and one failure with an average response time of 206 ms and throughput of 32.64 requests/second, whereas Run B contains 33 requests and no failures with an average response time of 236 ms and throughput of 3.70 requests/second. The values demonstrate how the interface supports comparative interpretation; they are not presented as a controlled benchmark experiment.

2.8 Containerization and Deployment Architecture

The backend and load-testing workflow were designed to run in a Linux-based containerized environment. Containerization separates the application runtime and its dependencies from the host system and provides a consistent execution unit for deployment.

K3s is used as the lightweight Kubernetes distribution for orchestration. In the development setup, K3s was prepared on Linux, while the final backend was moved to a cloud-hosted environment. This changes the operational model from a locally dependent development setup to a continuously accessible deployment environment.

The resulting deployment concept can be represented as:

Frontend → Cloud Backend → Asynchronous Test Execution → Locust → Results

The cloud deployment is particularly relevant for this project because the load-testing workload should not depend on the developer's local machine remaining available during a test.

2.9 CI/CD Workflow

The project also incorporates a CI/CD workflow using GitLab CI/CD and GitLab Runner. The purpose of this layer is to automate the process by which code changes are validated and the deployment workflow is executed.

A simplified workflow is:

1. A change is committed and pushed to the repository.
2. GitLab CI/CD detects the change and starts the configured pipeline.
3. The GitLab Runner executes the required jobs in a Linux-compatible environment.
4. The application/container is prepared and the deployment-related steps are executed.
5. The resulting service can then be accessed through the deployed environment.

This integration connects source-code management, automated execution, containerized deployment, and the running backend into one development workflow.

2.10 Technical Stack

Component	Technology
Backend	Python / Flask
Load-testing engine	Locust
Frontend	Web-based frontend application
Backend communication	HTTP / JSON APIs
Containerization	Docker
Container orchestration	K3s / Kubernetes
CI/CD	GitLab CI/CD
CI/CD execution	GitLab Runner
Deployment	Cloud-hosted Linux environment
Source control	Git / GitHub and GitLab workflow

Table 2.1: Elevate Load Tester technical stack

Implementation and Evaluation

3.1 Implementation

The implementation is organised around the sequence of actions performed by a user. The frontend does not directly configure the underlying load-testing infrastructure. Instead, it communicates with backend services that handle analysis, test creation, execution and result retrieval.

The major implementation components are:

1. Target submission and validation.
2. API/sample-input analysis.
3. Automated edge-case generation.
4. User-controlled edge-case selection.
5. Load configuration.
6. Asynchronous Locust test execution.
7. Test-status monitoring.
8. Result collection and display.
9. Historical test-run comparison.
10. Containerized and cloud-based deployment.

3.2 Target Analysis Workflow

The analysis workflow establishes whether the supplied target can be processed before a load test is started. The target is first sent to the backend, where basic checks are performed. The backend then selects the analysis behaviour appropriate to the target.

For API testing, the sample input becomes the source from which variants can be constructed. The analysis stage therefore creates a bridge between a human-readable request example and a machine-executable load-testing workload.

3.3 Test-Case Construction

The generated cases are represented in a form that can be selected by the frontend and later passed to the execution layer. The design separates case generation from case execution, which is important because the user should be able to review the generated workload before consuming compute resources on a load test.

The frontend evidence confirms that the cases are shown with descriptive labels such as “missing field”, “null value”, “type mismatch”, “boundary value”, and known attack-pattern variants. The selection interface also accepts a configurable user population, spawn rate, and duration.

3.4 Load-Test Execution

Once the workload has been confirmed, the backend initiates a Locust test session. The asynchronous design prevents the frontend request-response cycle from being tied to the full duration of the test. A separate test-session state can therefore be queried while the workload is executing.

The execution environment is containerized and deployed through the cloud-hosted backend infrastructure. K3s provides lightweight orchestration for the Linux environment used to run the services.

3.5 Result Processing

The result-processing stage transforms the output of the load test into information that can be consumed by the frontend. Rather than exposing raw engine output as the primary interface, the application presents aggregated metrics and per-case information.

This design makes the output understandable to a user who may not need to interact directly with Locust’s internal execution model. It also creates a common structure for comparing different test sessions.

3.6 Evaluation Methodology

The project was evaluated functionally through the complete workflow implemented in the frontend and backend. The evaluation focused on whether the system could progress through the intended stages rather than immediately invoking a raw load test.

The principal evaluation scenarios are:

1. Submit a target for analysis.

2. Provide or process sample API input.
3. Generate and display edge cases.
4. Select a subset of cases.
5. Configure users, spawn rate, and duration.
6. Start an asynchronous load test.
7. Observe the test status.
8. Inspect completed results.
9. Compare two completed runs.

3.7 Evaluation Metrics

The system exposes several metrics that are useful for assessing load-test behaviour.

3.7.1 Total Requests

Total requests represents the number of requests generated during the test session. It provides a basic measure of the workload actually executed.

3.7.2 Failures

The failure count records requests that did not complete successfully according to the load-testing engine. Comparing failures between test sessions can help identify whether a change in workload or configuration affects reliability.

3.7.3 Response Time

Average response time provides an overall indication of responsiveness. Minimum and maximum response times provide additional information about the spread of observed request latency.

3.7.4 Requests per Second

Requests per second provides a throughput-oriented measure of the workload handled during the test. It is useful when comparing test sessions with different configurations.

3.7.5 Duration

Test duration records the configured or observed execution period and provides context for interpreting the request count and throughput.

3.8 Experimental Configurations

The frontend allows the user to vary the test configuration rather than using a single fixed workload. In particular, the user can configure:

- Number of users / concurrent virtual-user population.
- Spawn rate.
- Test duration.
- Selected edge cases.

This makes it possible to conduct repeated experiments using the same target but different workload sizes or case selections.

3.9 Results

The supplied interface evidence demonstrates successful completion of the complete testing cycle. A sample completed run contains per-case request statistics and an overall total. The displayed example reports 294 total requests and one failure, with average, minimum and maximum response-time values available to the user.

The comparison interface further demonstrates that completed sessions can be evaluated together. In the provided example, one run used a 50-user configuration while another used a 15-user configuration. The displayed metrics allow the user to observe changes in request volume, failures, response time and throughput.

3.10 Analysis of Results

The sample results demonstrate the primary analytical value of the platform: a single test session can be examined at both aggregate and edge-case levels, while historical runs can be compared using the same metric definitions.

The comparison shown in Figure 2.5 also illustrates why workload configuration must be considered when interpreting performance numbers. A larger user population generated substantially more requests and higher throughput in the displayed

example, but the two runs should not be treated as equivalent controlled experiments because their configurations differ.

The system therefore provides the infrastructure required for repeatable experimentation, while the validity of a performance conclusion still depends on selecting appropriate and comparable experimental configurations.

3.11 Deployment Evaluation

The final system is not limited to a local development environment. The backend has been deployed on a cloud-hosted environment, while the frontend has also been deployed. This establishes the intended separation between the user-facing application and the backend/load-testing infrastructure.

The deployment also resolves an important limitation of a purely local K3s setup: the service can be accessed without requiring the developer's local machine or local Linux environment to remain active.

Conclusion

4.1 Summary of Work

The Elevate Load Tester project presents an end-to-end load-testing workflow in which automated input analysis is performed before workload generation. The system accepts a target, validates and analyses it, generates candidate edge cases, allows the user to select the desired cases, configures the workload, and starts an asynchronous Locust-based load test.

The frontend provides an accessible interface for these operations and presents completed results through aggregate metrics and per-case statistics. A comparison feature further supports evaluation of previous test sessions.

Behind the frontend, the project integrates a Flask backend, Locust load-testing engine, containerized execution, K3s-based orchestration, cloud deployment, and Git-Lab CI/CD. The combination demonstrates how an application can connect software-engineering practices with practical performance testing.

4.2 Key Findings

The major findings from the completed implementation are:

1. An input-analysis stage can be placed before load generation to make the testing workflow more structured and user-controlled.
2. Automatically generated edge cases provide a practical mechanism for broadening the workload beyond a single nominal request.
3. Asynchronous execution is appropriate for load tests because test sessions may continue for a significant period after the initial request has been submitted.
4. Separating the frontend from the load-testing engine makes the system easier to use and provides a clearer abstraction for users.
5. Cloud deployment removes dependence on the developer's local machine for continuous backend availability.
6. CI/CD connects repository changes with repeatable deployment operations and reduces the amount of manual infrastructure handling required.
7. Historical comparison improves the usefulness of individual test results by allowing users to evaluate changes across runs.

4.3 Limitations

The current project has several limitations that should be considered when interpreting its results.

1. The generated edge cases are based on the information available from the supplied target and sample input; they cannot guarantee complete coverage of all application-specific failure conditions.
2. Load-test results depend on the target system, network conditions, cloud resources, selected workload, and execution environment.
3. A displayed test result is not by itself sufficient to establish a statistically rigorous performance benchmark. Controlled repeated experiments are required for stronger conclusions.
4. The current platform focuses on the demonstrated edge-case categories and does not constitute a complete security-testing framework.
5. The available result comparison is metric-oriented; more advanced statistical analysis and visual trend analysis could provide additional insight.
6. Resource allocation and workload scaling can be further refined for substantially larger concurrent test populations.

4.4 Future Work and Recommendations

The platform can be extended in several directions:

1. **Advanced test-case generation:** Extend the input-analysis layer to infer more application-specific cases from schemas, documentation, or observed responses.
2. **Adaptive workload generation:** Automatically adjust workload intensity based on the target's response behaviour and configured resource limits.
3. **Richer result visualisation:** Add response-time distributions, percentile plots, throughput graphs, failure trends, and time-series views.
4. **Automated regression testing:** Allow a new run to be compared automatically against a baseline and flag significant changes.
5. **Scalable distributed execution:** Expand the orchestration layer to distribute large workloads across multiple test workers when required.

6. **Persistent experiment history:** Maintain structured records of targets, configurations, selected cases, and results for long-term analysis.
7. **Improved target support:** Extend analysis and crawling capabilities for a wider range of websites and API styles.
8. **Security-oriented separation:** Provide clearer separation between performance-oriented edge cases and dedicated security testing so that the platform can integrate safely with specialised security tools.

4.5 Final Remarks

Elevate Load Tester demonstrates a complete software-engineering approach to automated performance testing. Instead of treating load testing as a direct URL-to-Locust operation, the project introduces a structured analysis stage that helps construct a more meaningful workload before execution.

The completed workflow combines frontend interaction, backend processing, automated edge-case generation, asynchronous execution, result presentation, historical comparison, containerization, Kubernetes-based orchestration, cloud deployment, and CI/CD. This provides a foundation that can be extended toward more scalable and statistically rigorous performance-testing workflows.